

# **My learning of, experience with, and opinions on SPIFFE and the SPIRE implementation**

Ethan Gregg August Davenport

## **Preface**

When I began my research into SPIFFE/SPIRE, I had a cursory knowledge of network security and cybersecurity. My knowledge in the field mostly consisted of what I learned by default as a result of being a computer engineer, as well as my experience with Windows Defender, various hardware exploits I know of from jailbreaking multiple devices, and an incident when I had the private key for a Discord bot available on a public GitHub repository.

Digital key cryptography was probably the first concept I learned, specifically the asymmetric algorithmic encryption behavior of these keys. I learned of certificates shortly after, and while I have some gaps in knowledge of their operation, I believe I have a functional understanding of the certificate authority's role in signing certificates.

During my initial reading of the SPIFFE standard's documentation, the X.509 standard came up multiple times, and as such, I decided to add it to my knowledge pool. The X.509 standard defines the formatting of public key certificates, known for its use in multiple internet protocols, such as TLS/SSL (and by relation, HTTPS) and SPIFFE. An X.509 certificate contains a certificate version number, serial number, an ID for the signing algorithm, an issuer name, a validity period, a subject name, the public key info for the aforementioned subject, various optional fields, the certificate signature algorithm, and the certificate signature. The certificate effectively allows receivers to verify the identity of the sender and make sure its information has not been tampered with.

## **The Secret Zero and the Bottom Turtle**

"The Secret Zero Problem" describes a seemingly paradoxical situation, wherein for machines or tasks to get their first "secret", they need some way to prove their identity or have the secret be pre-assigned by administrators. This effectively means that for a task to gain the trust of something else, it must already have the trust of a higher authority. Most workloads today cannot prove their identity without pre-existing authority, and built-in static secrets are usually saved in persistent memory, and as such, are vulnerable to attacks from bad actors. "The Bottom Turtle Problem" describes a similar problem, wherein a security layer needs to be trusted by a more authoritative security layer, which in turn, also needs to be trusted by a higher security layer, and so on. The hypothetical highest security layer on which the trust of all things relies has been dubbed the "bottom turtle," based on an old anecdote revolving around the cosmos and a stack of turtles. These two similar problems are both claimed to be solved through the use of the SPIFFE standard and its implementation known as SPIRE.

## SPIFFE

The Secure Production Identity Framework For Everyone, colloquially known as SPIFFE, is a set of open source standards for identifying software systems, known as workloads, in a set of dynamic environments. SPIFFE systems are meant to easily and reliably authenticate what software is being run. The fundamental core of the SPIFFE standard is the SVID, a short-lived cryptographic identity document, used by workloads to communicate and prove a bearer's identity to other workloads. The documentation website for SPIFFE boasts a decent amount of services that use and support SPIFFE.<sup>1</sup>

The SPIFFE identity is the basic text component for identifying SPIFFE workloads, similar to a person's name. To be compliant with the standards of SPIFFE, a SPIFFE ID must follow the standards of an RFC 3986-compliant URL, even if it does not point to an actual website or such. The SPIFFE identity must have a scheme set to "spiffe", must have a trust domain, and must not include a fragment or query component. SPIFFE IDs must not exceed 2048 bytes in length, which all SPIFFE-compliant implementations must support.

The Trust Domain is a vague term that represents the individual or organization running their own independent SPIFFE infrastructure. The Trust domain is self-registered and has no delegating authority to assert and register a base domain to a real-world entity, as is done with DNS. The format of the Trust domain also has its own set of text restrictions, which can be seen in the attached notes document. Operators can choose any trust domain name they wish; however, operators are instructed to choose names that will avoid possible collisions.

The SPIFFE Verifiable Identity Document, frequently shortened to the SVID, is the identity document standard used by SPIFFE-compliant workloads. An SVID is deemed valid if it has been signed by an authority within the relevant trust domain. As such, a signing authority must exist in each trust domain and must carry an SVID of its own. The SPIFFE standard does not specify SVID as a format for documents; rather, the pre-existing X.509 and JWT standards are used. The key trait of SVIDs is their transient nature, where SVIDs are valid for a limited amount of time, after which they are then replaced by a new SVID. Because of this temporality, workloads should be given enough time to receive their new SVIDs before the previous identity document expires.

SPIFFE Bundles are bundles that contain a trust domain's public cryptographic keys. These bundles are used to prove the identity of the signing authority claimed by SVIDs. The contents of SPIFFE Bundles are expected to change over time, and in a similar nature to SVIDs, New Bundles should be distributed prior to the expiration of the previous Bundles. SPIFFE bundles take the format of JWK sets, not to be confused with the aforementioned JWT format.

---

<sup>1</sup> <https://spiffe.io/docs/latest/spiffe-about/overview/>

SPIFFE Bundle Maps are objects that contain a collection of SPIFFE Bundles; one may even refer to these as SPIFFE Bundle Bundles. SPIFFE Bundle Maps are JSON-encoded structures, wherein no duplicate domain name keys are allowed to exist. The primary purpose of these bundle maps is to enable cross-trust-domain authentication, which the SPIFFE standard reverse to as SPIFFE Federation.

The JWT SVID format exists, however, it is strongly advised to use the X.509-SVID format when possible. Outside of file encoding, the two formats have differentiating internal structures, specifics of which can be seen in the attached notes file and official documentation.<sup>2</sup>

### **SPIFFE Workload API**

The SPIFFE Workload API is not an actual application programming interface; rather, it is a guideline for valid SPIFFE APIs, such as the SPIRE implementation. The Workload API standard standardizes a way to retrieve and validate SPIFFE identities, served by the SPIFFE Workload Endpoint. Implementors are required to provide support for both X.509 SVIDs and JWT SVIDs in order to be compliant with the SVID standard.

The Workload Endpoint specification requires that direct client authorization is never done, and out-of-band authenticity checks must be done instead through various methods (such as UNIX domain sockets or Kubernetes platform metadata). This process is known as workload attestation. Clients of the implemented workload API should maintain an open connection as long as is responsibly possible; the connection may be terminated at any time by the server or client. If the connection is terminated, the client should attempt to immediately establish a new connection so that the workload retains an updated set of identity materials, such as its SVID or a SPIFFE bundle.

During my initial reading, I found the official documentation on Stream Responses to be somewhat confusing. I was initially under the impression that SPIFFE workloads must communicate exclusively through server-side streams, and as such, all data was restricted to the stream response format outlined by the documentation,<sup>3</sup> such as how all previously sent data must be included in new messages, either updated, redacted, or maintained as the same. Fortunately for SPIFFE implementers worried about bandwidth being taken up by unnecessary data, the outlined server-side streams are generally only used for the distribution of identity materials, such as SVIDs and SPIFFE bundles.

---

<sup>2</sup> <https://spiffe.io/docs/latest/spiffe-specs/x509-svid/>

<sup>3</sup> [https://spiffe.io/docs/latest/spiffe-specs/spiffe\\_workload\\_api/#43-stream-responses](https://spiffe.io/docs/latest/spiffe-specs/spiffe_workload_api/#43-stream-responses)

The SPIFFE Workload API specification also defines the specific outline for the code of how the API handles requests for SVIDs; the specification notably requires implementers to always support requests for JWT-SVIDs, a decision which I will comment on later in this document.

The Workload Endpoint page argues that portable and interoperable cryptographic identities for networked workloads are the primary use case for SPIFFE. In order for this to be achieved, a standardized way to retrieve and consume identity-related services was formed in the SPIFFE Workload Endpoint specification. The SPIFFE Workload Endpoint specification details how workloads may access identity-related services at runtime, and servers as the mechanism for initial identity bootstrapping. Workloads notably do not know their identity before bootstrapping, as such, the workload endpoint must be exposed through local endpoints as a Unix Domain Socket, or TCP listen socket, and must be served over gRPC. Clients can be configured explicitly with the socket location or use an environment variable.

The SPIFFE documentation also provides a specific list of error codes the workload Endpoint must respond with.<sup>4</sup>

### **SPIFFE Federation**

The SPIFFE Federation specifications define the documents and interfaces necessary for bridging systems with different trust domains. Through the usage of SPIFFE Bundles, workloads in different trust domains can verify the SVIDs of each other. While the existence of SPIFFE federation is understandable, it seems to contradict the previous requirements outlined by the SVID standard. The questionability of SPIFFE Federation is further raised through the “Security Considerations” section in its documentation;<sup>5</sup> notable security concerns include that configuration parameters for federation relationships are highly vulnerable to tampering and allow for a possible method to easily exploit and compromise the security of a SPIFFE network. It is also highlighted that SPIFFE Federation can be used to extend chains of trusts through passing SPIFFE bundles through multiple trust domains, which in turn allows for entire SPIFFE networks to be compromised if one trust domain is compromised. SPIFFE Federation is ultimately a confusing and questionable framework that will not be outlined in this document.

It should be restated that SPIFFE is not an implementation of anything, and is rather a specific outline for how implementations should behave. It can be seen as somewhat similar to a programming language, where the compiler is the implementation. Expecting a single entity to create an entire API for the sole purpose of implementing SPIFFE standards can be considered obtuse; as such, various implementations of SPIFFE already exist, including SPIRE, the primary implementation developed by the creators of the SPIFFE standard.

---

<sup>4</sup> [https://spiffe.io/docs/latest/spiffe-specs/spiffe\\_workload\\_endpoint/#appendix-a-list-of-error-codes](https://spiffe.io/docs/latest/spiffe-specs/spiffe_workload_endpoint/#appendix-a-list-of-error-codes)

<sup>5</sup> [https://spiffe.io/docs/latest/spiffe-specs/spiffe\\_federation/#7-security-considerations](https://spiffe.io/docs/latest/spiffe-specs/spiffe_federation/#7-security-considerations)

## **SPIRE**

Knowledge of the SPIFFE standard is ultimately not necessary to use SPIRE; however, it is highly recommended as it allows for an understanding of what SPIRE actually does.

The SPIFFE Runtime Environment, commonly known as SPIRE, is the implementation of the workload API outlined by SPIFFE standards. SPIRE further complicates the Server-workload relationship outlined in the SPIFFE standards by introducing the agent, a middleman between the workload and server. Deployments of SPIRE are composed of one central SPIRE server and one or more SPIRE agents.

The Server acts as the signing authority for identities in its registered domain, maintaining a registry of workload identities and the required conditions for workload identities to be issued. The server uses node attestation to identify agents automatically, and it creates SVIDs for workloads when the request is made by an authenticated agent. The server can be configured through various plugins, allowing for the node attestation method to be changed, changing the method of how registration entries and private keys are stored, and changing the certificate authority to an upstream identity, requiring SVIDs to be signed by a higher authority rather than the local SPIRE server.

The aforementioned “middleman”, the agent, exposes the implemented workload API to workloads, attests the identity of workloads that call the API, and provides workloads with their SVIDs. SPIRE Agents must be installed on each node on which a SPIRE workload is running. Agents, similar to the server, can be configured with various plugins, allowing for the workload attestation process to be changed, as well as key generation methods to be changed.

### **Registration & Attestation**

Registration entries must first be created by an operator before a workload can be identified by the server. Registration entries outline specifically how to identify workloads and what SPIFFE IDs to give them.

Attestation of the node and workload is done by gathering specific attributes of each; workload attestation is done after node attestation.

Node attestation is performed by the SPIRE server and the agent running on the node, through the interrogation of the node and its environment to find information that only the “real” node would own, such as an identity document given beforehand to the node, a private key stored on a hardware security module, a join token, or other credentials such as KSA tokens. Successful node attestation results in the agent being given a unique SPIFFE ID.

Node Attestors supported by SPIRE include and are currently limited to EC2 instances on AWS, VMs on Microsoft Azure, Compute Engine Instances on Google Cloud Platform, Nodes on a Kubernetes cluster, and a join token pre-shared between the agent and server, which expires immediately after use.

Information on how workload attestation is performed can be seen in the SPIRE official documentation.<sup>6</sup>

---

<sup>6</sup> <https://spiffe.io/docs/latest/spire-about/spire-concepts/#workload-attestation>

## My Experience with SPIFFE and SPIRE

After having read the relevant documentation on SPIFFE and SPIRE, my initial plan for a demo of the system was (what I believed to be) simple, displaying two SPIRE workloads establishing a secure connection between each other, while a third malicious workload is blocked from accessing data from these workloads.

This began with installing the SPIRE server and agent files for Linux from the page hosted on the official SPIFFE website.<sup>7</sup> Official documentation suggested installing to the root,<sup>8</sup> which would later turn out to be a bad decision for my use case. Configuration files of the server and agent were left the same as the default variables. Because of the aforementioned node attestation methods for SPIRE, I was limited to either creating Kubernetes clusters or using Token sharing. After briefly attempting to set up a Kubernetes cluster, I decided to use the token method. This notably invalidates the “secret zero solution” SPIRE claimed to be, as the agent needed that initial secret to establish trust with the server.

The SPIRE server started, and the token was generated. The agent was run in another command window, with the jointoken created by the server as a startup argument. The workload that I would run was registered to the server.

From here, I attempted to install the Spiffe library for Python; however, this would become a challenge. As a result of my installed version of Linux, pip could not install packages globally. This issue with the permissions of pip is usually not an issue, as it simply requires the creation of a Python virtual environment, from which pip can be used to install packages to that specific virtual environment. This did not work, as SPIRE was installed at the root of my system, as per the aforementioned suggestion. My next idea was to run pip with elevated permissions, but this also did not work, as the Python virtual environment would, in turn, needs elevated access to write anything to the directory where SPIRE was installed. My attempts to run the virtual environment with elevated permissions did not work either, so at this point, I attempted to override the security measures to stop pip from installing libraries globally.

This should have worked; however, attempting to do so caused dependency errors with other libraries, and as such, the library could not be installed globally. It was at this point that I decided to just move the install to my desktop directory, wherein pip worked normally in the virtual environment.

---

<sup>7</sup> <https://spiffe.io/downloads/#spire-releases>

<sup>8</sup> <https://spiffe.io/docs/latest/deploying/install-server/>

The endpoint socket for Spiffe was set as an environment variable inside the virtual environment, and I attempted to run my Python program. This did not work, as it turns out, the py-spiffe library had recently undergone a naming change, and instead of marking deprecated functions and classes as deprecated to allow older applications and the various examples on the internet to still run, they were mostly removed outright from the library, causing most online example programs to not run, as well as any program created by a large language model to not run.

While I was able to eventually create a simple Python workload that prints its SPIFFE ID to the terminal, I was unable to follow through with my initial plan, as I could not manage to have two SPIRE workloads communicate with each other using pypiffe.

### **Criticisms - Preface**

Despite the knowledge I have gained in the field over the past 4 weeks, I should disclose that I am in no means an expert in SPIFFE/SPIRE, let alone cybersecurity. As such, my criticism towards SPIFFE, SPIRE, and the SPIRE environment should not be taken completely at face value, as realistically, at least one of my criticisms is rooted in misunderstanding what I read, or ignorance in the field.

### **Criticisms of SPIRE**

The inability of my initial demo to come to fruition is something I blame primarily on the lack of documentation of py-spiffe. The official GitHub page for py-spiffe, maintained by tech giant Hewlett Packard Enterprise, contains only three example programs, all of which are slight deviations of each other and only display the retrieval of SVIDs. The included spiffe-tls package documentation, a specific package that exclusively establishes TLS connections using SPIFFE, inexplicably has significantly more information available than the primary package (and still only lists the usage and function of four functions).

This lack of resources on the Python library is something I cannot ultimately blame on the SPIFFE/SPIRE team, as the Python library is not officially part of the SPIFFE repository. However, it is indicative of a much more serious issue, in that the library implementations for programming languages are not all that good. The C & C++ libraries have not been updated since August 2021, and this repository only exists because an official C/C++ library developed by the SPIFFE/SPIRE team “became out-of-date and was archived.”<sup>9</sup> The C# library is not in the same dire state as its C++ brother, however, it is also severely lacking in documentation, and it appears that a large majority of its code was written by Claude.

---

<sup>9</sup> <https://spiffe.io/docs/latest/deploying/libraries/>



The only major languages that have decent SPIRE support appear to be Go, Java, and Rust, of which Go appears to have the best support.

Go's presence is not at all surprising, considering Google's heavy hand in the creation of SPIFFE and SPIRE, but it's similarly indicative of SPIRE's reliance on pre-existing major solutions. As previously mentioned, SPIRE only supports specific methods for Node attestation, and the only local methods are through using a pre-generated token (insecure) or with Kubernetes clusters. The other methods all rely on running services on AWS, Google Servers, or Microsoft Azure virtual machines, and ultimately cause SPIRE to be less attractive for use in local servers.

SPIRE itself is also technically complex, with Defakto claiming small-scale deployments of SPIRE can take 6-12 months to develop, while complex and large-scale deployments can take up to multiple years.<sup>10</sup> Because SPIRE is an implementation of SPIFFE, it also shares the concerns raised with the standard, such as the SPIRE server and agent being a weak point in the chain of trust.

### **Criticisms of SPIFFE**

The aforementioned security risks caused by the SPIFFE server apply to the outline given by the SPIFFE standard. Further issues with the standard are caused by its strict and sometimes questionable requirements, such as how support for JWT-SVIDs is required for implementations of the SPIFFE API, as well as SPIFFE Federation.

SPIFFE Federation is a concept I can't exactly understand either. From my perspective, one of SPIFFE's advantages is the closed network created by the trust domain. As such, allowing exterior, possibly compromised, workloads access to local workloads, effectively dissolving the trust domain, seems like a questionable decision, and I feel would have been better implemented through a separate method. This, of course, can be avoided by making implementations not compliant with the Spiffe standard.

My greatest criticism of SPIFFE comes in the form of its relationship to the Secret Zero Problem and the Bottom Turtle problem. SPIFFE implementations ultimately require clients to still possess something for the server to validate them, and as such, the client still must know what requirements it has to fulfill before being verified. While the absence of something more precise for verification is useful for situations, such as mass software deployments, it also raises security concerns about bad actors possibly being able to access a network with less friction than the secret token method.

---

<sup>10</sup> [https://www.defakto.security/blog/simplifying-spiffe-accessible-workload-identity-with-spirl/](https://www.defakto.security/blog/simplifying-spiffe-accessible-workload-identity-with-spir/)

As far as SPIFFE and the Bottom Turtle, I do not comprehend how SPIFFE is supposed to solve the problem of the Bottom Turtle. SPIFFE Servers ultimately still rely on a Certificate Authority, and by relation, the clients also rely on this Certificate Authority. Proof that this Certificate Authority can be trusted is still required to be proven by an entity with higher authority. Ultimately, SPIFFE servers still require pre-existing trust to exist, which brings me to my personal beliefs on trust in cybersecurity.

### **The Bottom Turtle Cannot Be Solved**

I believe that the trust system in cybersecurity is ultimately based on the human system of trust. I can only verify my identity to other people through an identity signed by an entity with a higher level of trust (such as the New Jersey government). Mutual trust in the fact that the New Jersey government is, as it claims, not a secret, invalid group running the state, is maintained through the United States Government, and, depending on one's life outlook, the identity of the United States Government is verified by the UN, some higher being, or perhaps one thinks the identity of the United States Government is maintained by an invalid entity.

My point is, even in real life, proof of identity and trust is ultimately built on the trust of a larger entity, wherein the validity of one trust turtle is verified by a larger trust turtle, also on the back of another turtle, and so forth. Someone can't fully prove who they are without their DNA being sequenced, which can probably still be spoofed, not to mention that the person being identified must already trust the person identifying them. I believe this applies to computer processes as well, but in the computer instance, the DNA sequencing is instead a program reading the machine code of another program, and the program being read has to trust that the reader is not secretly malicious software that will steal important information by reading the machine code.

I believe that establishing trust between two workloads cannot be solved algorithmically and instead must be done on a case-by-case basis, where, like in real life, a person (workload) has to take an educated guess to determine if another unknown person (the outside workload) is trustworthy, and considering the rapid advancement in machine learning, this may become a more possible method in the coming years.

### **Praise for SPIFFE**

If I am to discuss what I like about SPIFFE, I cannot ignore the temporality of SVIDs. In a world where parallel processing rapidly advances, keys will eventually become easier to crack. It is this reason why I think SPIFFE's best "feature" is that SVIDs, along with their associated public and private keys, are frequently replaced. This frequent cycling that causes brute-force methods for key cracking to be invalid as methods for bypassing trust systems, as well as possible defense against private keys being leaked. SPIFFE's ability to invalidate potentially compromised CAs in its trust environment also proves to be an additional security measure against malicious attacks. SPIFFE's ability to dynamically create trust networks stands out as one of its best applications.

### **Alternatives**

SPIFFE is ultimately not the only solution for dynamic trust networks. Machine learning solutions have begun development, such as Deep Reinforcement Learning and Graph Neural Networks. SPIFFE itself is also an open source standard, and as such, possible vulnerabilities or unwanted behavior can be solved by forking the standard and defining new behavior.

It should also be noted that SPIRE is not the only implementation of SPIFFE. HashiCorp, a major player in cloud infrastructure, has Consul as a SPIFFE implementation. Defakto claims to be a simpler and more straightforward implementation of SPIFFE compared to SPIRE. Teleport also seems to be simpler, allowing for easier setup in pre-existing workflows.

### **Closing Thoughts**

SPIFFE presents itself as a solution to the Secret-Zero problem, acting as a framework for zero-trust networks, as well as providing an outline for secure, rapid deployment of multiple workloads of similar traits. Although some requirements by the standard are questionable, its system of frequent certificate cycling allows for heightened security in a dynamic environment. SPIRE, the standard implementation of SPIFFE, has shown promise for achieving the claims of SPIFFE; however, the steep learning curve, reliance on pre-existing architecture, and fractured libraries make it nonideal. Despite its shortcomings, SPIRE and SPIFFE exist as a valid method to reduce the problems caused by the Secret-Zero problem, and at worst, provide a solid framework for finding other methods to do so.